Chapter 1
Introduction

In this chapter we discuss first the main objectives of numerical analysis and introduce some basic notions. We investigate different sources of errors in scientific computation, define the notion of stability of a mathematical problem or a numerical algorithm, the time and memory resources needed to perform the algorithm. We study the computer representation of integer and real numbers, and present some numerical problems due to the finite-digit arithmetic.

1.1. The Main Objective and Notions of Numerical Analysis

In scientific computations the first step is the mathematical modeling of the physical process. This is the task of the particular scientific discipline (physics, chemistry, biology, economics, etc.). The resulting model frequently contains parameters, constants, initial data which are typically determined by observations or measurements. If the mathematical model and its parameters are given, then we can use it to answer some questions related to the physical problem. We can ask qualitative questions (Does the problem have a unique solution? Does the solution have a limit as the time goes to infinity? Is the solution periodic? etc.), or we can ask quantitative questions (What is the value of the physical variable at a certain time? What is the approximate solution of the model?). The qualitative questions are discussed in the related mathematical discipline, but the quantitative questions are the main topics of numerical analysis. The main objective of numerical analysis is to give exact or approximate solutions of a mathematical problem using arithmetic operations (addition, subtraction, multiplication and division). See Figure 1.1 below for the schematic steps of the scientific computation of physical processes.

The numerical value of a physical quantity computed by a process described in Figure 1.1 is, in general, not equal to the real value of the physical quantity. The sources of error we get is divided into the following two main categories: *inherited error* and *computational error*. The mathematical modeling is frequently a simplification of the physical reality, so we generate an inherited error when we replace the physical problem by a mathematical model. This kind of error is called *modeling error*. An other subclass of the inherited error is what we get when we determine the parameters of the mathematical model by measurements, so we use an approximate parameter value instead of the true one. This is called *measurement error*.

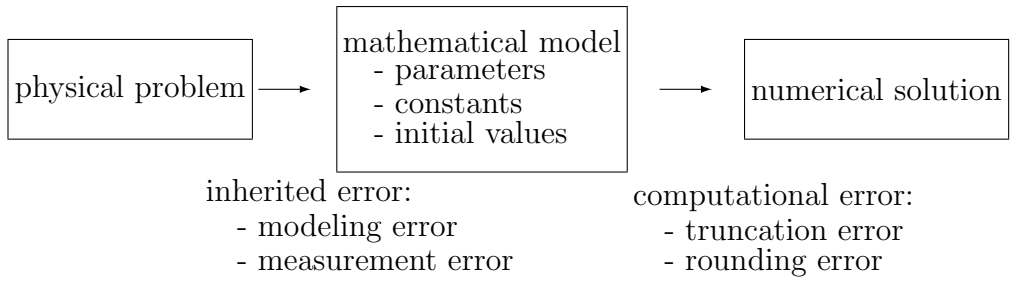

Figure 1.1: Scientific computations

The computational error is divided into two classes: *truncation error* and *rounding error*. We get a truncation error when we replace the exact value of a mathematical expression with an approximate formula.

Example 1.1. Suppose we need to compute the value of the function $f(x) = \sin x$ at a certain argument x. We can do it using arithmetic operations if instead of the function value $f(x)$ we compute, e.g., its Taylor-polynomial around 0 of degree 5: $T_5(x) = x - x^3/3! + x^5/5!$. The Taylor's theorem (Theorem 2.5 below) says that if $f(x)$ is replaced by $T_5(x)$, then the resulting error has the form $\frac{f^{(6)}(\xi)}{6!}x^6 = -\frac{\sin\xi}{6!}x^6$, where ξ is a number between 0 and x. This is the truncation error of the approximation, which is small if x is close to 0. □

The rounding error appears since real numbers can be stored in computers with finite digits accuracy. Therefore, we almost always generate a rounding error when we store a real number in a computer. Also, after computing each arithmetic operations, the computer rounds the result to a number which can be stored in the computer (see Sections 1.2–1.4).

When we specify a numerical algorithm, the first thing we have to investigate is the truncation error, since a numerical value is useful only if we know how large is the error of the approximation. The next notion we discuss related to a numerical algorithm is the *stability*. This notion is used in two meanings in numerics. We can talk about the *stability of a mathematical model* or about the *stability of a numerical method*. First we consider an example.

Example 1.2. Consider the linear system

$$\begin{aligned} 8x + 917y &= 1794 \\ 7x + 802y &= 1569. \end{aligned}$$

Its exact solution is $x = -5$ and $y = 2$. But if we change the coefficient of the variable x in the second equation to 7.01, then the solution of the corresponding system

$$8x + 917y = 1794$$
$$7.01x + 802y = 1569$$

is $x = -1.232562589$ and $y = 1.967132499$ (up to 9 decimal digits precision). We observe that 0.14% change in the size of a single coefficient results in 75.3% and 1.6% changes in the solutions, rspectively. □

We say that a mathematical problem is *correct* or *stable* or *well-conditioned*, if a "small" change in the parameters of the problem results only in "small" change in the solution of the problem. In the opposite case we say that the problem is *incorrect* or *ill-conditioned* or *unstable problem*. The linear system in the previous example is an incorrect mathematical problem.

We say that a numerical algorithm is *stable with respect to rounding errors* if the rounding errors do not influence the result of the computation significantly. If the computed result is significantly different from the true value, then we say that the *algorithm is unstable*. Next we present an example of an an unstable algorithm.

Example 1.3. Consider the following three recursive sequences:

$$\begin{aligned} x_n &= \frac{1}{3}x_{n-1}, \qquad x_0 = 1, \\ y_n &= 2y_{n-1} - \frac{5}{9}y_{n-2}, \qquad y_0 = 1, \quad y_1 = \frac{1}{3}, \\ z_n &= \frac{13}{3}z_{n-1} - \frac{4}{3}z_{n-2}, \qquad z_0 = 1, \quad z_1 = \frac{1}{3}. \end{aligned} \tag{1.1}$$

It is easy to show that all the three recursions generate the same sequence $x_n = y_n = z_n = \frac{1}{3^n}$, i.e., the three sequences are algebraically equivalent. But in practice, the numerical computations of the three recursions give different results. In Table 1.1 the first 18 terms of the sequences are displayed. The computations are performed using single precision floating point arithmetic in order to enlarge the effect of the rounding errors. We observe that the sequence x_n produce the numerical values of $1/3^n$, but the numerical values of y_n and z_n are different from it due to the accumulation of the rounding error. Both sequences has rounding error, but for the sequence z_n the error increases so rapidly, that in the 18th term it is of order 10^2. In this case the numerical values of z_n do not converge to 0. We experience that the sequence x_n is a stable method, but z_n is an unstable method to compute the values of $1/3^n$.

To check that the errors we observed in the previous computation are the consequence of the rounding errors, we repeated the generation of the three sequences but now using a double precision floating point arithmetic. We present here the error of the 18th terms: $|y_{18} - 1/3^{18}| = -2.5104e-13$ and $|z_{18} - 1/3^{18}| = 2.3804e-07$. We can observe that the magnitude of the errors are much smaller in this case. □

In case of an algorithm which terminates in a finite number of steps we are usually interested in the *time complexity* or the *cost* of an algorithm. By this we mean the number of steps, or more precisely, the *number of arithmetic operations* needed to perform the algorithm. Consider first an example.

Example 1.4. Evaluate numerically the polynomial $p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10$ at the point x. Certainly, we can do it using the formula of p literally. It contains 4 additions/subtractions, 4

Table 1.1:

n	x_n	y_n	$\lvert y_n - 1/3^n \rvert$	z_n	$\lvert z_n - 1/3^n \rvert$
2	0.111111	0.111111	2.2352e-08	0.111111	4.4703e-08
3	0.037037	0.037037	4.0978e-08	0.037037	1.8254e-07
4	0.012346	0.012346	6.9849e-08	0.012346	7.3109e-07
5	0.004115	0.004115	1.1688e-07	0.004118	2.9248e-06
6	0.001372	0.001372	1.9465e-07	0.001383	1.1699e-05
7	0.000457	0.000458	3.2442e-07	0.000504	4.6795e-05
8	0.000152	0.000153	5.4071e-07	0.000340	1.8718e-04
9	0.000051	0.000052	9.0117e-07	0.000800	7.4872e-04
10	0.000017	0.000018	1.5019e-06	0.003012	2.9949e-03
11	0.000006	0.000008	2.5032e-06	0.011985	1.1980e-02
12	0.000002	0.000006	4.1721e-06	0.047920	4.7918e-02
13	0.000001	0.000008	6.9535e-06	0.191674	1.9167e-01
14	0.000000	0.000012	1.1589e-05	0.766693	7.6669e-01
15	0.000000	0.000019	1.9315e-05	3.066773	3.0668e+00
16	0.000000	0.000032	3.2192e-05	12.267091	1.2267e+01
17	0.000000	0.000054	5.3653e-05	49.068363	4.9068e+01
18	0.000000	0.000089	8.9422e-05	196.273453	1.9627e+02

multiplications and 3 exponentials. The exponentials mean 3+2+1=6 number of multiplications, i.e., altogether 10 multiplications are needed to apply the formula of p. But we can rewrite p as follows:

$$p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10 = (((5x - 8)x + 2)x + 4)x - 10.$$

This form of the polynomial requires only 4 additions/subtractions and 4 multiplications. □

The previous method can be extended to polynomials of degree n:

$$a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0 = ((\cdots((a_n x + a_{n-1})x + a_{n-2})x + \cdots)x + a_1)x + a_0$$

This formula requires only n additions/subtractions and n multiplications. This way of organizing a polynomial evaluation is called *Horner's method*. The method can be defined by the following algorithm.

Algorithm 1.5. Horner's method

INPUT: n - degree of the polynomial
$a_n, a_{n-1}, \ldots, a_0$ - coefficients of the polynomial
x - argument
OUTPUT: y - function value of the polynomial at the argument x

$y \leftarrow a_n$
for $i = n-1, \ldots, 0$ **do**
  $y \leftarrow yx + a_i$
end do
output(y)

The execution of a multiplication or division requires more time than that of an addition or subtraction. Therefore, in numerical analysis, we count the number of multiplications and divisions separately to the number of additions and subtractions.

It is also important to know the *space complexity* of an algorithm, which is the amount of the memory storage needed in the worst case at any point in the algorithm. When we work with an algorithm to solve a linear system with a 10×10 coefficient matrix, the storage cannot be a problem. But the same with a 10000×10000 dimensional matrix can be problematic. In case of algorithms working with a big amount of data, we prefer a method which requires less amount of memory space. For example, if in a matrix nonzero elements appear only in the main diagonal and in some diagonals above and below, then it is practical to use an algorithm which utilizes the special structure of the data, and does not store the unnecessary zeros in the matrix during the computation. We will see such methods in Section 3.5 below.

1.2. Computer Representation of Integers and Reals

Let I be a positive integer with a representation in base b number system with m number of digits:

$$I = (a_{m-1}a_{m-2}\dots a_1a_0)_b, \qquad \text{where} \quad a_i \in \{0, 1, \dots, b-1\}.$$

Its value is

$$I = a_{m-1}b^{m-1} + a_{m-2}b^{m-2} + \dots + a_1b + a_0.$$

Therefore, the largest integer can be represented with m digits is $I_{\max}$ where all digits equal to $b-1$. Its numerical value is

$$I_{\max} = (b-1)(b^{m-1} + b^{m-2} + \dots + b + 1) = b^m - 1.$$

Hence on m digits we can represent (store) integers from 0 up to $b^m - 1$, which is b^m number of different integers.

Suppose we use a base 2, other words, *binary* number system. Then on m bits we can store 2^m number of integers. We describe two methods to store negative integers. The first method is the *sign-magnitude representation*. Here we allocate a *sign bit* (typically the *most significant bit*, i.e., the left-most bit), which is 0 for positive integers and 1 for negative integers. Then on the rest of the $m-1$ bits we can store the magnitude or absolute value of the number. Then $I_{\max} = 2^{m-1} - 1$ is the largest, and $I_{\min} = -I_{\max}$ is the smallest integer which can be represented. In this system the integer 0 can be stored as an identically 0 bit sequence or as $100\dots0$.

Example 1.6. In Table 1.2 we listed all the integers which can be represented on $m = 3$ bits using the sign-magnitude binary representation. □

In practice the *two's-complement representation* is frequently used to store signed integers. Let I be an integer which we would like to represent on m bits. Instead of I we store the binary form of the number C defined by

$$C = \begin{cases} I, & \text{if } 0 \le I \le 2^{m-1} - 1, \\ 2^m + I, & \text{if } -2^{m-1} \le I < 0. \end{cases}$$

Table 1.2: Sign-magnitude binary representation on $m = 3$ bits

I	a binary code
0	000
1	001
2	010
3	011
0	100
-1	101
-2	110
-3	111

Here the largest and the smallest representable integer is $I_{\max} = 2^{m-1} - 1$ and $I_{\min} = -2^{m-1}$, respectively. Therefore, if $0 \le I \le 2^{m-1} - 1$, then $C < 2^{m-1}$, i.e., the first bit of C is 0. On the other hand, if $-2^{m-1} \le I < 0$, then it is easy to see that $2^{m-1} \le C \le 2^m - 1$, i.e., the first bit of C is 1.

An important advantage of the two's-complement representation is that the subtraction can be obtained as an addition (see Exercise 4).

Example 1.7. Table 1.3 contains all the integers which can be represented on $m = 3$ bits using the two's-complement binary representation. □

Table 1.3: Two's-complement representation on $m = 3$ bits

I (in decimal)	I (in binary)	C, the stored binary
0	000	000
1	001	001
2	010	010
3	011	011
-1	-001	111
-2	-010	110
-3	-011	101
-4	-100	100

Next we discuss the representation of real numbers. We recall that the real number in base b number system

$$x = (x_{m-1}x_{m-2}\cdots x_0.x_{-1}x_{-2}\cdots)_b, \qquad x_i \in \{0, 1, \ldots, b-1\}$$

has the numerical value

$$x = x_{m-1}b^{m-1} + x_{m-2}b^{m-2} + \cdots x_1 b + x_0 + \frac{x_{-1}}{b} + \frac{x_{-2}}{b^2} + \cdots = \sum_{i=-\infty}^{m-1} x_i b^i.$$

Consider the real number 126.42. Different books define the normal form of this number as $1.2642 \cdot 10^2$ or $0.12642 \cdot 10^3$. In this lecture notes we use the first form as the normal form. Therefore, the *normal form* of a real number $x \neq 0$ in a base b number system is $x = \pm m \cdot b^k$, where $1 \le m < b$. The number m is called the *mantissa*, and k the *exponent* of the number. In order to represent a real number, or in other words, a *floating*

point number we write it in a normal form in a base b number system, and we would like to store its signed mantissa and exponent. Computers use different number of bits to store these numbers. Here we present an IEEE specification[1] to represent floating point numbers on 32 bits (the so-called *single precision*), and on 64 bits (the *double precision*) using the binary number system. This representation is used in IBM PCs. Consider the binary normal form $x = (-1)^s m \cdot 2^k$, where $s \in \{0, 1\}$ and $m = (1.m_1 m_2 m_3 \ldots)_2$. The value s is stored in the 1st bit. Instead of the exponent k, we store its shifted value, the nonnegative integer $e = k + 127$ on bits 2–9. In our definition of the binary normal form, a nonzero x has a mantissa of the form $m = (1.m_1 m_2 \ldots)_2$, i.e., it always starts with 1, which we do not store, we store the fractional digits of the mantissa rounded to 23 bits. These 23 bits are stored on bits 10–32 of the storage. This IEEE specification defines the representation of the number 0, and introduces two special symbols, `Inf` (to store infinity as a possible value) and `NaN` (not-a-number) in the following way:

number	s	e (bits 2–9)	mantissa bits (bits 10–32)
$+0$	0	00000000	every mantissa bit=0
-0	1	00000000	every mantissa bit=0
$+$`Inf`	0	11111111	at least one mantissa bit=0
$-$`Inf`	1	11111111	at least one mantissa bit=0
$+$`NaN`	0	11111111	every mantissa bit=1
$+$`NaN`	1	11111111	every mantissa bit=1

The symbol `Inf` can be used in programs as a result of a mathematical operation with value ∞, and the symbol `NaN` can be a result of a mathematical operation which is undefined (e.g., a division by 0 or a root of a negative number in real numbers). Both symbols can be positive or negative. The definition yields that the exponent $e = (11111111)_2 = 255$ is used exclusively for the special symbols `Inf` and `NaN`. For finite reals the possible values are $0 \le e \le 254$, hence the possible values of the exponent k are $-127 \le k \le 127$. Therefore, the smallest positive representable number corresponds to exponent $k = -127$ and mantissa $(1.00\ldots01)_2$. Hence its value is $x_{\min} = (1 + 1/2^{23})2^{-127} \approx 10^{-38}$. The largest real can be stored is $x_{\max} = (1.11\ldots1)_2 2^{127} = (2 - 2^{-23})2^{127} \approx 10^{38}$.

The representation on 64 bits is similar: the shifted exponent $e = k + 1023$ is stored on bits 2–12, the fractional part of the mantissa is stored on bits 13–64. Then the range of real numbers which can be stored in the computer is, approximately, from 10^{-308} to 10^{308}.

Example 1.8. Suppose we would like to store reals on 4 bits using a binary normal form. For example, we use the 1st bit as the sign bit, the shifted exponent $e = k + 1$ is stored on the 2nd bit, and the fractional part of the mantissa is stored on bits 3–4. (The symbols `Inf` and `NaN` are not defined now.) The nonnegative real numbers which can be represented by the above method are listed in Table 1.4, and are illustrated in Figure 1.2.

□

We can see that, using any floating point representation, we can store only finitely many reals on a computer. The reals which can be stored without an error in a certain floating point representation are called *machine numbers*. The machine number which is

[1] IEEE Binary Floating Point Arithmetic Standard, 754-1985.

Table 1.4: Nonnegative reals on 4 bits.

s	e	m	x
0	0	00	0
0	0	01	$(1.01)_2 \cdot 2^{-1} = (1+\frac{1}{4})\frac{1}{2} = \frac{5}{8}$
0	0	10	$(1.10)_2 \cdot 2^{-1} = (1+\frac{1}{2})\frac{1}{2} = \frac{3}{4} = \frac{6}{8}$
0	0	11	$(1.11)_2 \cdot 2^{-1} = (1+\frac{1}{2}+\frac{1}{4})\frac{1}{2} = \frac{7}{8}$
0	1	00	$(1.00)_2 \cdot 2^{0} = 1 = \frac{8}{8}$
0	1	01	$(1.01)_2 \cdot 2^{0} = 1+\frac{1}{4} = \frac{10}{8}$
0	1	10	$(1.10)_2 \cdot 2^{0} = 1+\frac{1}{2} = \frac{12}{8}$
0	1	11	$(1.11)_2 \cdot 2^{0} = 1+\frac{1}{2}+\frac{1}{4} = \frac{7}{4} = \frac{14}{8}$

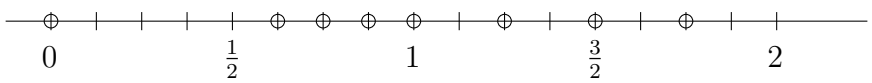

Figure 1.2: Nonnegative machine numbers on 4 bits.

stored in a computer instead of the real number x is denoted by $\mathrm{fl}(x)$. If $|x|$ is smaller than the smallest positive machine number, then, by definition, $\mathrm{fl}(x) = 0$, and if $|x|$ is larger than the largest positive machine number, then we define $\mathrm{fl}(x) = \texttt{Inf}$ for $x > 0$ and $\mathrm{fl}(x) = -\texttt{Inf}$ for $x < 0$. In the first case we talk about arithmetic *underflow*, and in the second case, about arithmetic *overflow*. The definition of $\mathrm{fl}(x)$ in the intermediate cases can be different in different computers. There are basically two methods: In the first case we take the binary normal form of x, consider its mantissa $m = (1.m_1m_2m_3\ldots)_2$, and we consider as many first several mantissa fractional bits as it is possible to store in the particular representation. We store them, and omit the rest of the mantissa bits. This method is called *chopping* of the mantissa. For example, using the single precision representation defined above, we store the first 23 mantissa fractional bits.

The other method, the *rounding*, is more frequently used to define the mantissa bits of the machine number $\mathrm{fl}(x)$. Here the mantissa of $\mathrm{fl}(x)$ is defined so that $\mathrm{fl}(x)$ be the nearest machine number to x. In case when x is exactly an average of two consecutive machine numbers, we could round down or up. The IEEE specification for single precision representation we defined above uses the following rule: Let the normal form of a positive real x be $x = m2^k$, where $m = (1.m_1m_2\ldots m_{23}m_{24}\ldots)_2$. Let $x' = (1.m_1m_2\ldots m_{23})_2 2^k$ and $x'' = \big((1.m_1m_2\ldots m_{23})_2 + 2^{-23}\big)2^k$. Then x' and x'' are consecutive machine numbers, $x' \le x \le x''$ and $x'' - x' = 2^{k-23}$. The specification defines

$$\mathrm{fl}(x) = \begin{cases} x', & \text{if } |x-x'| < \frac{1}{2}|x''-x'|, \\ x'', & \text{if } |x-x''| < \frac{1}{2}|x''-x'|, \\ x', & \text{if } |x-x'| = \frac{1}{2}|x''-x'| \text{ and } m_{23} = 0, \\ x'', & \text{if } |x-x'| = \frac{1}{2}|x''-x'| \text{ and } m_{23} = 1. \end{cases}$$

In the critical case, i.e., if $|x-x'| = \frac{1}{2}|x''-x'|$, approximately half of the cases we round down and in the other cases we round up. An other reason for this definition is that in

this critical case the last mantissa bit is always 0, so a division by 2 can be performed on $\mathrm{fl}(x)$ without an error. Using the rounding, the error is

$$|x - \mathrm{fl}(x)| \le \frac{1}{2}|x'' - x'| = \frac{1}{2}2^{-23}2^k.$$

If we compare it to the exact value we get

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \le \frac{|x - \mathrm{fl}(x)|}{(1.m_1m_2\ldots)_2 \cdot 2^k} \le \frac{1}{2}2^{-23}.$$

We can see that the first machine number which is larger than 1 is $1 + 2^{-23}$ in the single precision floating point arithmetic. Let ε_{m} denote the difference of the first machine number right to 1 and the number 1. This number is called *machine epsilon*. Therefore, ε_{m} is the smallest power of 2 (in a binary storage system) for which the computer evaluates the inequality $1 + \varepsilon_{\mathrm{m}} > 1$ to be true. The following theorem can be proved similarly to the consideration above for the single precision floating point representation.

Theorem 1.9. *Let* $0 < \mathrm{fl}(x) < \texttt{Inf}$*, and suppose the floating point representation uses rounding. Then*

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \le \frac{1}{2}\varepsilon_m.$$

The proof of the next result is left for Exercise 5.

Theorem 1.10. *Suppose we use a number system with base* b*, and let* t *be the number of mantissa bits in the floating point representation. Then*

$$\varepsilon_m = \begin{cases} 2^{-t}, & \text{if } b = 2, \\ b^{1-t}, & \text{if } b \ne 2. \end{cases}$$

Now we define the notion of the error of an approximation and other related notions. Let x be a real number, and consider $\tilde{x}$ as its approximation. Then the *absolute error* or just simply the *error* of the approximation is the number $|x - \tilde{x}|$. Frequently the error without knowing the magnitude of the numbers does not mean too much. For example, 10000.1 can be considered as a good approximation of 10000, but in general, 1.1 is not considered as a good approximation of 1, but in both cases the errors are the same, 0.1. We may get more information if we compare the error to the exact value. The *relative error* is defined by

$$\frac{|x - \tilde{x}|}{|x|} \qquad (x \ne 0).$$

We say that in a base b number system $\tilde{x}$ is *exact in* n *digits* if

$$\frac{|x - \tilde{x}|}{|x|} \le \frac{1}{2}b^{1-n}.$$

We can see that the smaller is the relative error, the larger is the number of exact digits. In a decimal number system ($b = 10$) we can formulate the relation between the relative error and the number of exact digits in the following way: if the relative error decreases by a factor of 1/10, then the number of exact digits increases by 1.

Example 1.11. Let $x = 1657.3$ and $\tilde{x} = 1656.2$. Then the absolute error is $|x - \tilde{x}| = 1.1$, and the relative error is $|x - \tilde{x}|/x = 0.0006637$ (with 7 decimal digits precision). Since $|x - \tilde{x}|/x = 0.0006637 < 0.5 \cdot 10^{-2}$, the approximation is exact in 3 digits. On the other hand, if x is approximated by the value $\tilde{x} = 1656.9$, then $|x - \tilde{x}|/x = 0.0002413 < 0.5 \cdot 10^{-3}$, and hence the approximation is exact in 4 digits. □

The previous definition and Theorem 1.9 yield that the single precision floating point arithmetic is exact in 24 binary digits. Usually, we are interested in the exact number of digits in a decimal number system. In case of a single precision floating point arithmetic, we get it if we find the largest integer n for which

$$\frac{1}{2}2^{-23} \leq \frac{1}{2}10^{1-n}.$$

It can be computed that $n = 7$ is the number of exact digits for a single precision floating point arithmetic.

Example 1.12. Consider $x = 12.4$. First rewrite it in binary form: $12 = (1100)_2$. Find the binary form of its fractional part:

$$0.4 = (0.x_1x_2x_3\ldots)_2 = \frac{x_1}{2} + \frac{x_2}{2^2} + \frac{x_3}{2^3} + \cdots .$$

If we multiply 0.4 by 2, then its integer part gives x_1. $0.4 \cdot 2 = 0.8$, hence $x_1 = 0$. Consider the fractional part of this product, 0.8, and we repeat the previous procedure. $0.8 \cdot 2 = 1.6$, so $x_2 = 1$. The fractional part of the product is 0.6, which gives: $0.6 \cdot 2 = 1.2$, and therefore, $x_3 = 1$. The fractional part of the product is 0.2. We have $0.2 \cdot 2 = 0.4$, hence $x_4 = 0$, and we continue with 0.4. We can see that the digits 0011 will be repeated periodically infinitely many times, i.e., $0.4 = (0.01100110011001100110011001 1\ldots)_2$. The binary normal form of x is

$$x = 12.4 = (1.10001100110011001100110011001 1\ldots)_2 \cdot 2^3.$$

Rounding the mantissa to 23 bits (down) we get

$$\mathrm{fl}(x) = (1.10001100110011001100110)_2 \cdot 2^3.$$

Its numerical value (in a decimal form) is $\mathrm{fl}(x) = 12.3999996185302734375$. □

The arithmetic operations performed by a computer can be defined as

$$\begin{aligned}
x \oplus y &:= \mathrm{fl}(\mathrm{fl}(x) + \mathrm{fl}(y)),\\
x \ominus y &:= \mathrm{fl}(\mathrm{fl}(x) - \mathrm{fl}(y)),\\
x \odot y &:= \mathrm{fl}(\mathrm{fl}(x) \cdot \mathrm{fl}(y)),\\
x \oslash y &:= \mathrm{fl}(\mathrm{fl}(x) / \mathrm{fl}(y)).
\end{aligned}$$

Here we always take the machine representation of each operands and also the result of the arithmetic operation.

In later examples we will use the so-called *4-digit rounding arithmetic* or simply *4-digit arithmetic*. By this we mean a floating point arithmetic using a decimal number system with 4 stored mantissa digits (and suppose we can store any exponent). This means that, in every step of a calculation, the result is rounded to the first 4 *significant digits*, i.e., from the first nonzero digits for 4 digits, and this rounded number is used in the next arithmetic operation. We can enlarge the effect of rounding errors in such a way.

Example 1.13. Using a 4-digit arithmetic we get $1.043 + 32.25 = 33.29$, and similarly, $1.043 \cdot 32.25 = 33.64$ (after rounding). But $1.043 + 20340 = 20340$, since we rounded the exact value 20341.043 to 4 significant digits. □

Exercises

1. Convert the following decimal numbers to binary form:
$$57, \quad -243, \quad 0.25, \quad 35.27$$
2. Convert the binary numbers to decimal form:
$$(101101)_2, \quad (0.10011)_2, \quad (1010.01101)_2$$
3. Show that the two's-complement representation of a negative integer can be computed in the following way: Take the binary form of the absolute value of the number. Change all 0's to 1's and all 1's to 0's, and add 1 to the resulting number.
4. Let I_1 and I_2 be two positive integers with m bits. Show that $I_1 - I_2$ can be computed if we first consider the two's-complements representation C_2 of I_2, add I_1 to it, and finally, take the last m bits of the result.
5. Prove Theorem 1.10.
6. Write a computer code which gives back the machine epsilon of the particular computer.
7. Compute the exact number of digits of a machine number in case of a double precision floating point arithmetic.
8. Let $x = (x_0.x_1x_2\ldots x_mx_{m+1}x_{m+2}\ldots)\cdot 10^k$, $\tilde{x} = (x_0.x_1x_2\ldots x_m\tilde{x}_{m+1}\tilde{x}_{m+2}\ldots)\cdot 10^k$, i.e., x and $\tilde{x}$ has the same order of magnitude, and its first $m+1$ digits are the same. Show that, in this case, $\tilde{x}$ is an approximation of x with at least m number of exact digits.

1.3. Error Analysis

Let x and y be positive real numbers, and consider the numbers $\tilde{x}$ and $\tilde{y}$ as an approximation of x and y. Let $|x - \tilde{x}| \le \Delta_x$ and $|y - \tilde{y}| \le \Delta_y$ be the error bounds of the approximation. The relative error bounds are denoted by $\delta_x := \Delta_x/x$ and $\delta_y := \Delta_y/y$, respectively. In this section we examine the following question: We would like to perform an arithmetic operation (addition, subtraction, multiplication or division) on the real numbers x and y, but instead of it, we perform the operation on the numbers $\tilde{x}$ and $\tilde{y}$

(suppose without an error). We will consider this latter number as an "approximation" of the original one. We will examine the error and the relative error of this "approximation".

Consider first the addition. We are looking for error bounds Δ_{x+y} and δ_{x+y} such that

$$|x+y-(\tilde{x}+\tilde{y})| \le \Delta_{x+y} \quad \text{and} \quad \frac{|x+y-(\tilde{x}+\tilde{y})|}{x+y} \le \delta_{x+y}.$$

Theorem 1.14. *The numbers*

$$\Delta_{x+y} := \Delta_x + \Delta_y \qquad \textit{and} \qquad \delta_{x+y} := \max\{\delta_x, \delta_y\}$$

are absolute and relative error bounds of the addition, respectively.

Proof. Using the triangle inequality and the definitions of Δ_x and Δ_y, we get

$$|x+y-(\tilde{x}+\tilde{y})| \le |x-\tilde{x}| + |y-\tilde{y}| \le \Delta_x + \Delta_y.$$

This means that $\Delta_x + \Delta_y$ is an upper bound of the error of the addition.

Using the above relation, we obtain

$$\begin{aligned}\frac{|x+y-(\tilde{x}+\tilde{y})|}{x+y} &\le \frac{\Delta_x+\Delta_y}{x+y} \\ &= \frac{x}{x+y}\delta_x + \frac{y}{x+y}\delta_y \\ &\le \max\{\delta_x, \delta_y\}.\end{aligned}$$

Therefore, $\max\{\delta_x, \delta_y\}$ is a relative error bound of the addition. □

Clearly, the above theorem can be generalized for addition of several numbers: the error bounds will be added, and the relative error bound is the maximum value of the relative error bounds. We can reformulate this result as follows: the number of exact digits of the approximation of the sum is at least the smallest of the number of exact digits of the approximations of the operands. Certainly, the theorem gives the worst case estimate. In practice the errors can balance each other. For example, let $x = 1$, $y = 2$, $\tilde{x} = 1.1$ and $\tilde{y} = 1.8$. Then $x + y = 3$ and $\tilde{x} + \tilde{y} = 2.9$. Therefore, the error of the sum is only 0.1, smaller than the sum of the error of the terms, 0.3.

Theorem 1.15. *Let $x > y > 0$. The numbers*

$$\Delta_{x-y} := \Delta_x + \Delta_y \qquad \textit{and} \qquad \delta_{x-y} := \frac{x}{x-y}\delta_x + \frac{y}{x-y}\delta_y$$

are absolute and relative error bounds of the subtraction.

Proof. The inequalities

$$|x-y-(\tilde{x}-\tilde{y})| \le |x-\tilde{x}| + |y-\tilde{y}| \le \Delta_x + \Delta_y$$

imply the first statement. Consider

$$\frac{|x-y-(\tilde{x}-\tilde{y})|}{x-y} \leq \frac{\Delta_x+\Delta_y}{x-y} = \frac{x}{x-y}\delta_x + \frac{y}{x-y}\delta_y,$$

which gives the second statement. □

We can observe that if we subtract two nearly equal numbers, then the relative error can be magnified compared to the relative error of the terms. In other words, the number of exact digits can be significantly less that in the original numbers. This phenomenon is called *loss of significance*.

Example 1.16. Let $x = 12.47531$, $\tilde{x} = 12.47534$, $y = 12.47326$ and $\tilde{y} = 12.47325$. Then $\delta_x = 2.4 \cdot 10^{-6}$ and $\delta_y = 8 \cdot 10^{-7}$. On the other hand, $x - y = 0.00205$, $\tilde{x} - \tilde{y} = 0.00209$, and so $\delta_{x-y} = 0.0195$. We can check that $\tilde{x}$ and $\tilde{y}$ are exact in 6 digits, but $\tilde{x} - \tilde{y}$ is exact only in 2 digits. □

Theorem 1.17. *Let $x, y > 0$. The numbers*

$$\Delta_{x\cdot y} := x\Delta_y + y\Delta_x + \Delta_x\Delta_y, \qquad \textit{and} \qquad \delta_{x\cdot y} := \delta_x + \delta_y + \delta_x\delta_y$$

are absolute and relative error bounds of the multiplication, respectively.

Proof. The triangle-inequality and simple algebraic manipulations yield

$$\begin{aligned}
|xy - \tilde{x}\tilde{y}| &= |xy - x\tilde{y} + x\tilde{y} - \tilde{x}\tilde{y}| \\
&\leq x|y-\tilde{y}| + |\tilde{y}||x-\tilde{x}| \\
&\leq x\Delta_y + |\tilde{y}|\Delta_x \\
&= x\Delta_y + |y+\tilde{y}-y|\Delta_x \\
&\leq x\Delta_y + y\Delta_x + \Delta_x\Delta_y,
\end{aligned}$$

hence the first statement is proved. Therefore, we get

$$\frac{|xy-\tilde{x}\tilde{y}|}{xy} \leq \frac{x\Delta_y + y\Delta_x + \Delta_x\Delta_y}{xy} = \delta_x + \delta_y + \delta_x\delta_y,$$

which implies the second statement. □

Since, in general, Δ_x and Δ_y are much smaller than x and y, and so $\Delta_x\Delta_y$ is much smaller than $x\Delta_y$ and $y\Delta_x$, we have that $x\Delta_y + y\Delta_x$ is a good approximation of the absolute error of the multiplication. Similarly, $\delta_x + \delta_y$ is a good approximation of the relative error of the multiplication. Both results mean that the errors do not propagate rapidly in multiplication.

Theorem 1.18. *Suppose $x, y > 0$ and $\delta_y < 1$. Then the numbers*

$$\Delta_{x/y} := \frac{x\Delta_y + y\Delta_x}{y(y-\Delta_y)} \qquad \text{and} \qquad \delta_{x/y} := \frac{\delta_x + \delta_y}{1-\delta_y}$$

are absolute and relative error bounds of the division, respectively.

Proof. Elementary manipulations give

$$\left|\frac{x}{y} - \frac{\tilde{x}}{\tilde{y}}\right| = \frac{|x\tilde{y} - xy + xy - \tilde{x}y|}{y|\tilde{y}|} \le \frac{x\Delta_y + y\Delta_x}{y|\tilde{y}|} = \frac{x\Delta_y + y\Delta_x}{y|y-(y-\tilde{y})|}.$$

Assumption $\delta_y < 1$ implies $|y-\tilde{y}| \le \Delta_y < y$, hence $|y-(y-\tilde{y})| \ge y - |y-\tilde{y}| \ge y - \Delta_y > 0$ proves the first statement.

For the second part, consider

$$\frac{\left|\frac{x}{y} - \frac{\tilde{x}}{\tilde{y}}\right|}{\frac{x}{y}} = \frac{|x(\tilde{y}-y) - y(\tilde{x}-x)|}{x|\tilde{y}|} = \frac{\left|\frac{\tilde{y}-y}{y} - \frac{\tilde{x}-x}{x}\right|}{\left|1 - \frac{y-\tilde{y}}{y}\right|} \le \frac{\delta_x + \delta_y}{1-\delta_y}.$$

□

If δ_y is small, then the relative error bound of the division can be approximated well by $\delta_x + \delta_y$. Similarly, if Δ_y is much smaller than y, then $\frac{1}{y}\Delta_x + \frac{x}{y^2}\Delta_y$ is a good approximation of $\Delta_{x/y}$. If y is much smaller than x, or if y is close to 0, then Δ_y or Δ_x can be significantly magnified, so the absolute error can be much larger than the absolute error of the terms.

Exercises

1. Let $x = 3.50$, $y = 10.00$, $\tilde{x} = 3.47$, $\tilde{y} = 10.02$. Estimate the absolute and relative error of
$$3x + 7y, \quad \frac{1}{y}, \quad x^2, \quad y^3, \quad \frac{4xy}{x+y}$$
(without evaluating the expressions) assuming we replace x and y by $\tilde{x}$ and $\tilde{y}$. Then compute the expressions numerically and compute the absolute and relative errors exactly. Compare them with the estimates.
2. Let $\tilde{x}$ be an approximation of x, and $|x - \tilde{x}| \le \Delta_x$. Let $f : \mathbb{R} \to \mathbb{R}$ be a differentiable function satisfying $|f'(x)| \le M$ for all $x \in \mathbb{R}$. Let $y = f(x)$ and consider $\tilde{y} = f(\tilde{x})$ as an approximation of y. Estimate the absolute error of the approximation. (Hint: Use the Lagrange's Mean Value Theorem.)

1.4. The Consequences of the Floating Point Arithmetic

Example 1.19. Solve the equation

$$x^2 - 83.5x + 1.5 = 0$$

using 4-digit arithmetic in the computations.

Using the quadratic formula and the 4-digit arithmetic we get the numerical values

$$\tilde{x} = \frac{83.5 \pm \sqrt{83.5^2 - 4 \cdot 1.5}}{2} = \frac{83.5 \pm \sqrt{6972 - 6.000}}{2} = \frac{83.5 \pm 83.46}{2},$$

hence

$$\tilde{x}_1 = \frac{167.0}{2} = 83.50, \quad \text{and} \quad \tilde{x}_2 = \frac{0.040}{2} = 0.020.$$

We can check that the exact solutions (up to several digits precision) are $x_1 = 83.482032$ and $x_2 = 0.0179679$. Using the relative error bounds for each roots we get $\delta_1 = 0.0002152$ and $\delta_2 = 0.113096$. The first root is exact in 4 digits, but the second is only in 1 digits. So there is a significant difference between the order of the magnitudes of the relative errors. What is the reason of it? In the computation of the second root, we subtracted two close numbers. This is the point where we significantly lost the accuracy. □

Consider the second root of $ax^2 + bx + c = 0$:

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \tag{1.2}$$

When b is negative and $4ac$ is much smaller than b^2, then we subtract two nearly equal numbers, and we observe the loss of significance. (This happened for the second root in Example 1.19.) To avoid this problem, consider

$$x_2 = \frac{b^2 - (b^2 - 4ac)}{2a(-b + \sqrt{b^2 - 4ac})} = \frac{2c}{-b + \sqrt{b^2 - 4ac}}. \tag{1.3}$$

This formula is algebraically equivalent to formula (1.2). But the difference is that here we do not subtract two close numbers (in the denominator we add two positive numbers). If b is positive, then for the first root we get

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}}. \tag{1.4}$$

Example 1.20. Compute the second root of the equation of Example 1.19 using 4-digit arithmetic and formula (1.4).

$$\tilde{x}_2 = \frac{2 \cdot 1.5}{83.5 + \sqrt{83.5^2 - 4 \cdot 1.5}} = \frac{3}{83.5 + 83.46} = \frac{3}{167.0} = 0.01796.$$

The relative error of x_2 is now $\delta_2 = 0.00044$, hence the exact number of digits is 4. □

Example 1.21. Suppose we need to evaluate the expression $\cos^2 x - \sin^2 x$. If $x = \frac{\pi}{4}$, then the exact value of this expression is 0, hence if x is close to $\frac{\pi}{4}$, then in the expression we need to subtract to nearly equal numbers, so we can face loss of significance. We can avoid it if, instead of the original formula, we evaluate its equivalent form, $\cos 2x$. □

In the previous examples we used algebraic manipulations to avoid the loss of significance. Now we show different techniques.

Example 1.22. Consider the function $f(x) = e^x - 1$. In the neighborhood of $x = 0$ we again need to subtract two nearly equal numbers, but here we cannot use an algebraic identity to avoid it. But here we can consider the Taylor series of the exponential function, and we get

$$f(x) = x + \frac{x}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots .$$

It is worth to take a finite approximation of this infinite series, and use it as an approximation of the function value $f(x)$. □

The next example shows a different problem.

Example 1.23. Evaluate the number $y = 20^{50}/50!$. The problem is the following: If we compute the numerator and the denominator separately first, then we run into the problem of overflowing the calculation if we use single precision floating point arithmetic. On the other hand, we know that $a^n/n! \to 0$ as $n \to \infty$, so the result must be a small number. We rearrange the computation as follows:

$$\frac{20^{50}}{50!} = \frac{20}{50} \cdot \frac{20}{49} \cdot \frac{20}{48} \cdots \frac{20}{1}.$$

Here in each steps the expressions we need to evaluate belong to the range which can be stored in the computer. This formula can be computed with a simple **for** cycle:

$y \leftarrow 20$
for $i = 2, \ldots, 50$ **do**
    $y \leftarrow y \cdot \frac{20}{i}$
end do
output(y)

The result is 3.701902 (with 7-digits precision). □

Example 1.24. Compute the sum

$$A = 10.00 + 0.002 + 0.002 + \cdots + 0.002 = 10.00 + \sum_{i=1}^{10} 0.002$$

using a 4-digit arithmetic. We perform the additions from left to right, so first we need to compute $10.00 + 0.002$. But with a 4-digit arithmetic the result is $10.00 + 0.002 = 10.002 = 10.00$ after the rounding. Adding the next number to it, because of the rounding to 4 digits, we get again $10.00 + 0.002 + 0.002 = 10.00$. Hence we get the numerical result $A = 10.00$.

Consider the same sum, but in another order:

$$B = 0.002 + 0.002 + \cdots + 0.002 + 10.00 = \sum_{i=1}^{10} 0.002 + 10.00.$$

First we need to compute $0.002 + 0.002 = 0.004$. The result is exact even if we use 4-digit arithmetic. Then we can continue: $0.002 + 0.002 + 0.002 = 0.006$ etc., and finally, $\sum_{i=1}^{10} 0.002 =$

0.02. Therefore, the numerical result will be $B = 10.02$. Here we have not observed any rounding error, since we could compute the result in each step exactly.

This example demonstrates that the addition using a floating point arithmetic is not a commutative operation numerically. □

A conclusion of the previous example is that in computing sums with several terms, it is advantageous to do the computation in an increasing order of the terms, since in that case we have a better chance for that the terms have similar order of magnitude, so the loss of significance has less chance.

Exercises

1. Investigate that in the next example what are the cases when we can observe the loss of significance. How can we avoid it?
   (a) $\ln x - 1$,
   (b) $\sqrt{x+4} - 2$,
   (c) $\sin x - x$,
   (d) $1 - \cos x$,
   (e) $(1 - \cos x)/\sin x$,
   (f) $(\cos x - e^{-x})/x$,
2. Compute the next expression using a 4-digit arithmetic $2.274 + 12.04 + 0.4233 + 0.1202 + 0.2204$, and then sort the terms in an increasing way, and repeat the calculation.